Building a Movie Recommendation System Using Collaborative Filtering

EECS 2502 Course Project - Winter 2025

By:

Hossein Rajabi
Shami-uz Zaman

April 6, 2025

Table of Contents

1. Introduction

1.1. Motivation

In today's digital world, many people feel overwhelmed by the large amount of available content, especially in areas like movie streaming. Recommendation systems help by suggesting items that match user preferences. This benefits both the users, who discover content more easily, and the platform, which often sees higher engagement. Our project focuses on building such a system for movies, using data on user behavior.

1.2. Project Objectives

The main goal of this project was to build and evaluate a movie recommendation system based on the MovieLens dataset. Our specific objectives were to:

- Load and clean the MovieLens 1M dataset, including ratings, movie details, and user demographics.
- Conduct Exploratory Data Analysis (EDA) to find interesting patterns and characteristics.
- Implement an item-based collaborative filtering model using the K-Nearest Neighbors (KNN) algorithm.
- Use K-Fold Cross-Validation to determine the optimal K and improve the model's accuracy.
- Measure accuracy using Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE).
- Generate lists of similar movies and personalized recommendations for selected users.
- Present our methods, findings, and limitations in a final report and Jupyter Notebook.

1.3. Project Scope and Adaptation

We originally planned to use the MovieLens 10M dataset and Apache Spark. However, our local machines could not handle the longer processing times. To ensure we could finish all project phases, we switched to the 1M dataset (one million ratings), which is still a recognized benchmark. We adapted our implementation to Python with `pandas` and `scikit-learn` for data operations and modeling. This approach let us do thorough EDA, hyperparameter tuning, and result analysis within our time and resource limits.

2. Dataset Description

2.1. Source and Overview

We used the MovieLens 1M dataset, provided by the GroupLens Research Project at the University of Minnesota [1]. It contains 1,000,209 ratings for approximately 3,900 movies, created by 6,040 users who joined MovieLens in 2000.

2.2. Data Files

The dataset has three main files that use `"::"` as a delimiter:

- `ratings.dat`
  Contains each rating in the format `UserID::MovieID::Rating::Timestamp`.
- `users.dat`
  Contains user demographics in the format `UserID::Gender::Age::Occupation::Zip-code`.
- `movies.dat`
  Contains movie information in the format `MovieID::Title::Genres`.

2.3. Key Characteristics

- **Users:** There are 6,040 users (ID range 1–6040). Demographics include gender (M or F), age group (7 categories), and occupation (21 categories).
- **Movies:** There are 3,883 distinct movies in `movies.dat` (ID range 1–3952). Among these, 3,706 received at least one rating. Each entry shows the title (with release year) and one or more genres.
- **Ratings:** There are 1,000,209 total ratings on a 1-to-5-star scale (only whole stars). Each user has at least 20 ratings. Timestamps are recorded in seconds since the epoch.
- **Sparsity:** The user-item matrix is sparse, with about 95.5% of possible ratings missing. This is typical in real-world recommendation data.

We loaded the data into `pandas` DataFrames. Basic cleaning steps included removing duplicates, converting timestamps to dates, and checking for valid ranges. Figures A1 to A10 in the Appendix show the main data distributions.

3. Exploratory Data Analysis

We examined the dataset to understand rating distributions, movie popularity, and user demographics:

- **Rating Distribution:** Most ratings are positive, with four-star ratings being the most common. One-star and two-star ratings are less frequent. The average rating is about 3.58 (Figure A1).
- **Ratings per Movie:** A few popular movies have thousands of ratings, while many have fewer. The median count is 124 (Figure A2).
- **Ratings per User:** Most users rate a moderate number of movies (median of 96), though some rate over 1,000. Each user has at least 20 ratings (Figure A3).
- **Genre Distribution:** Drama and Comedy are the largest genres, followed by Action, Thriller, and Romance. Film-Noir, Documentary, and War have high average ratings, while Horror is lower (Figures A6 and A7).
- **User Demographics:** About 72% of users are male (Figure A8). The largest age group is 25–34 (Figure A9). Average ratings vary slightly by gender and age (Figures A11 and A12).
- **Temporal Trends:** Rating activity peaked in 2000, with a small decrease in the average rating from 2000 to 2002 (Figure A13).

These findings suggest that the dataset is suitable for collaborative filtering, despite its sparseness and somewhat positive rating bias.

4. Methodology

4.1. Collaborative Filtering Approach

We used a collaborative filtering (CF) method to predict user ratings and build recommendations. CF relies on the idea that users with similar past preferences will likely agree again. It does not require detailed item information (e.g., genres, actors) because it focuses on user-item rating patterns.

4.2. Item-Based K-Nearest Neighbors (KNN)

We specifically implemented an **item-based** KNN. It finds similarities between movies based on their rating vectors. The steps are:

1. **Item Representation:** Each movie is represented as a vector across all users, with entries corresponding to user ratings. Given the large user base and high sparsity, these are often stored in a sparse matrix.
2. **Item Similarity:** We use cosine similarity. If $\mathbf{r}_i$ and $\mathbf{r}_j$ are rating vectors for movies i and j, then
$$\text{sim}(i, j) = \frac{\mathbf{r}_i \cdot \mathbf{r}_j}{\|\mathbf{r}_i\| \|\mathbf{r}_j\|}.$$
Similarities near 1 mean users rated both movies in a similar way; values near 0 indicate little overlap in rating patterns.

3. **Neighborhood Selection:** For each movie, we use `NearestNeighbors` (`metric='cosine'`) from `scikit-learn` to find the top K similar movies.

4. **Rating Prediction:** If user u has not rated movie i, we look at neighbor movies $N(i;u)$ that u has rated. The predicted rating is

$$\hat{r}_{ui} = \frac{\sum_{j \in N(i;u)} \text{sim}(i,j)\, r_{uj}}{\sum_{j \in N(i;u)} |\text{sim}(i,j)|}.$$

   If u has no ratings among those neighbors, we fall back on the user's average rating or the global average.

4.3. Recommendation Generation

To suggest movies for a user, we predict the ratings for the user's unrated movies and sort them in descending order of the predicted score. The top N are presented as recommendations.

5. Implementation and Evaluation

We built the system in a Jupyter Notebook using `pandas`, `NumPy`, `SciPy`, and `scikit-learn`. We used `Matplotlib` (and some `Seaborn` styling) for visualization.

5.1. Data Preparation

- **ID Mapping:** We mapped all UserIDs and MovieIDs to zero-based indices for easier handling in Python.
- **Sparse Matrix Construction:** We turned the user-item data into a `scipy.sparse.csr_matrix` of size 6040×3706. For the item-based model, we transposed it into a 3706×6040 matrix (movies by users).

5.2. Cross-Validation for Hyperparameter K

We tuned K (the number of neighbors) with 5-Fold Cross-Validation:

- We used `KFold` to create 5 folds, each time training on 4 folds and validating on the remaining fold.
- We built item-based KNN models on the training set and tested various K values (15, 20, 25, 30, 35, 40, 50).
- We predicted ratings for each user-movie pair in the validation set, then computed RMSE and MAE.

From Figure A14, performance generally improved with larger K, and $K = 50$ yielded the lowest RMSE.

5.3. Metrics

We used the following metrics:

$$\text{RMSE} = \sqrt{\frac{1}{|\mathcal{T}|} \sum_{(u,i)\in\mathcal{T}} (r_{ui} - \hat{r}_{ui})^2}, \quad \text{MAE} = \frac{1}{|\mathcal{T}|} \sum_{(u,i)\in\mathcal{T}} |r_{ui} - \hat{r}_{ui}|.$$

Here, $\mathcal{T}$ is the set of test ratings, and r_{ui} and $\hat{r}_{ui}$ are the real and predicted ratings.

5.4. Final Evaluation

After choosing $K = 50$:

1. **Cross-Validation Averages:** The mean RMSE was about 0.94, and the mean MAE about 0.73.
2. **Train/Test Split:** We also used a single 80/20 split. The RMSE was 0.94 and MAE was 0.73 on the test set, very close to the CV averages.

We trained the final model (K=50) on the full 1M dataset.

6. Results and Discussion

6.1. Predictive Accuracy

With $K = 50$, the RMSE is about 0.94, meaning predictions deviate by less than one star (on average) on the 5-star scale. This matches typical baseline KNN results for MovieLens 1M.

Figures for the single train/test split show:

- **Actual vs. Predicted (Figure A15):** Ratings cluster near the overall average (about 3.5–4.0), with fewer extreme predictions.
- **Distribution Comparison (Figure A16):** Actual ratings are discrete; predicted ratings form a smoother distribution centered near 3.8.
- **Residuals (Figure A17):** Errors form a near-normal curve around zero, indicating minimal bias. Most errors lie within ±1.5 stars.

6.2. Item Similarity Examples

Using the final model:

- **"Toy Story (1995)" (Figure A18):** Close neighbors include *Toy Story 2*, *Aladdin*, and *A Bug's Life*.
- **"Matrix, The (1999)" (Figure A19):** Similar titles are mainly sci-fi or action hits like *Terminator 2* and *Total Recall*.

6.3. Personalized User Recommendations

We generated sample recommendations for certain users:

- **User 1 (Figure A20):** Suggestions include *Cat from Outer Space*, *Beefcake*, and *7th Voyage of Sinbad*.
- **User 100 (Figure A21):** Suggestions include *Breaking the Waves* and *Chungking Express*.
- **User 5440 (Figure A22):** Suggestions include *Old Man and the Sea* and *Grace of My Heart*.

6.4. System Usability

Our Jupyter Notebook demonstrates a working pipeline but is mainly a proof-of-concept. A real deployment would need a user-friendly interface or API. Nevertheless, the system meets our goal of building, tuning, and evaluating an item-based KNN recommender.

7. Limitations

- **Sparsity:** Neighborhood-based methods like KNN can struggle with sparse data, though MovieLens 1M is moderately dense by industry standards.
- **Cold-Start Problem:** Our system does not address situations where new users or items have almost no ratings.
- **Scalability:** A brute-force KNN approach can be slow for extremely large datasets, requiring more advanced or distributed methods.
- **Popularity Bias:** KNN tends to recommend popular movies that already have many ratings.
- **Algorithm Scope:** We only used item-based KNN; user-based KNN or matrix factorization could behave differently.
- **Parameter Sensitivity:** We tuned K but kept the cosine metric. Other distance metrics might give different results.

8. Conclusion and Future Work

8.1. Conclusion

We built and analyzed an item-based KNN recommender using the MovieLens 1M dataset. Our EDA revealed important rating trends and user behaviors. We identified $K = 50$ as optimal via 5-Fold Cross-Validation, achieving about 0.94 RMSE, which is typical for a baseline KNN approach on this dataset. The final system can produce similar-movie lists

and user-specific recommendations. Although we initially considered MovieLens 10M and Apache Spark, we successfully adapted our plans to the 1M dataset and Python libraries.

8.2. Future Work

Future enhancements could include:

- **Other Algorithms:** Try matrix factorization or user-based KNN for comparison.
- **Hybrid Methods:** Combine rating-based CF with content features to reduce cold starts.
- **Cold-Start Solutions:** Implement techniques for new users or new items that have very few ratings.
- **Scalability:** Use approximate nearest neighbor libraries or distributed systems for larger datasets.
- **Evaluation Metrics:** Include precision/recall@N, NDCG, and other ranking-based metrics.
- **User Interface:** Develop a web interface or API for real-time recommendations.
- **Time Dynamics:** Account for shifting preferences over time.

9. Team Contributions

Hossein Rajabi was responsible for cleaning and preprocessing the dataset, conducting the Exploratory Data Analysis (EDA), and identifying the optimal model.
Shami-uz Zaman was responsible for implementing the item-based KNN model and performing the evaluation steps.

They both worked together on creating the presentation and writing the report.
For the presentation, **Shami** selected a suitable template and designed the slides, while **Hossein** decided on the specific content to include.
For the report, **Hossein** wrote the initial draft, and **Shami** handled proofreading and final edits before submission.

References

[1] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems (TiiS)* 5, 4, Article 19 (December 2015), 19 pages. http://dx.doi.org/10.1145/2827872

[2] MovieLens 1M Dataset. GroupLens Research. Retrieved from https://grouplens.org/datasets/movielens/1m/

Appendix: Figures

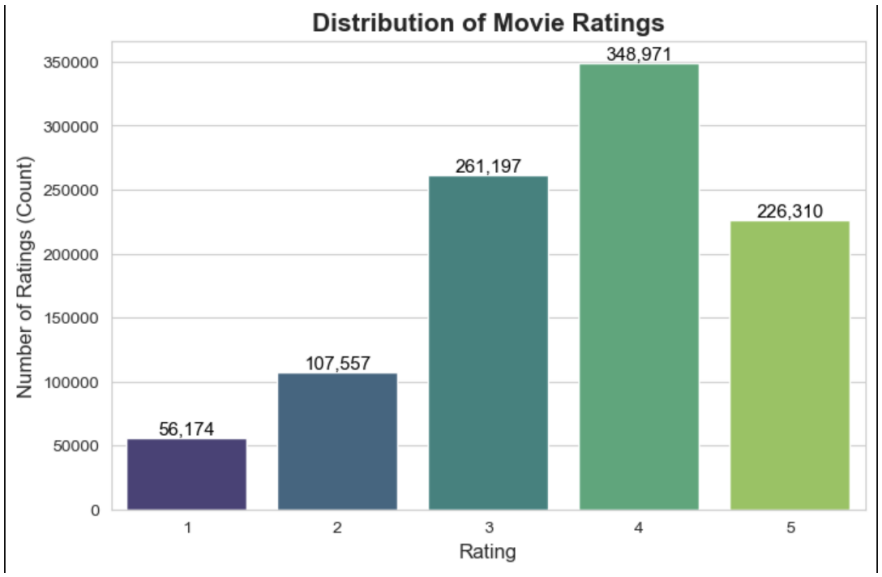

Figure A1: Distribution of Movie Ratings (1-5 Stars).

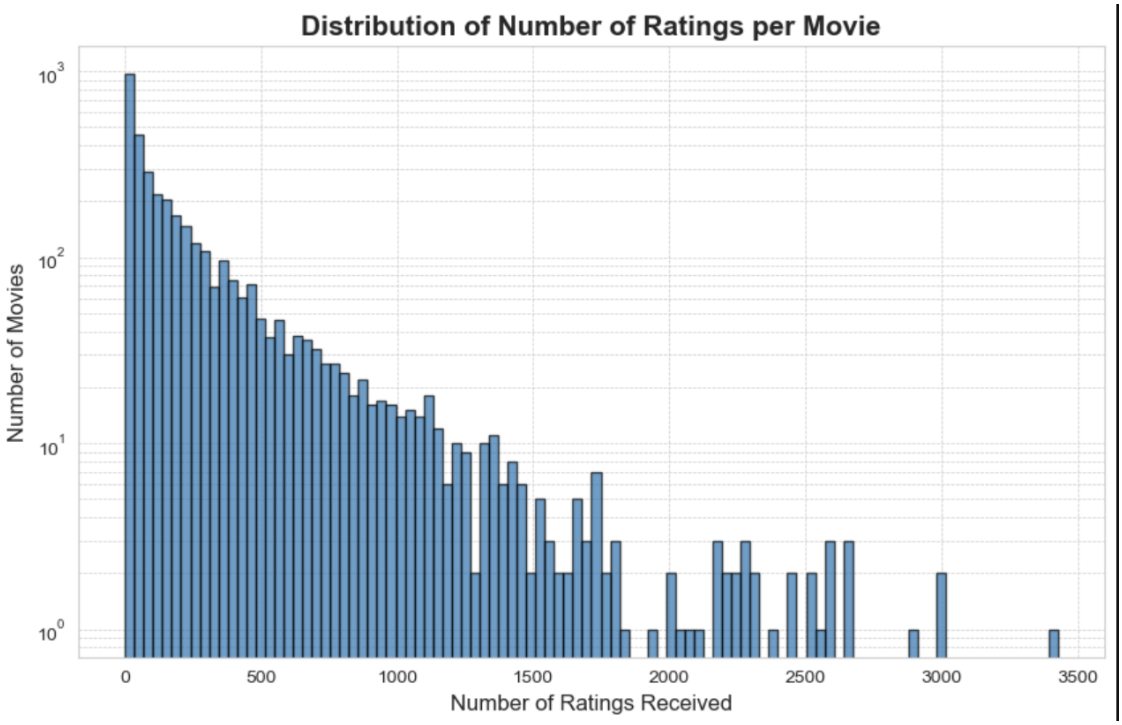

Figure A2: Distribution of Number of Ratings Received per Movie (Log Scale).

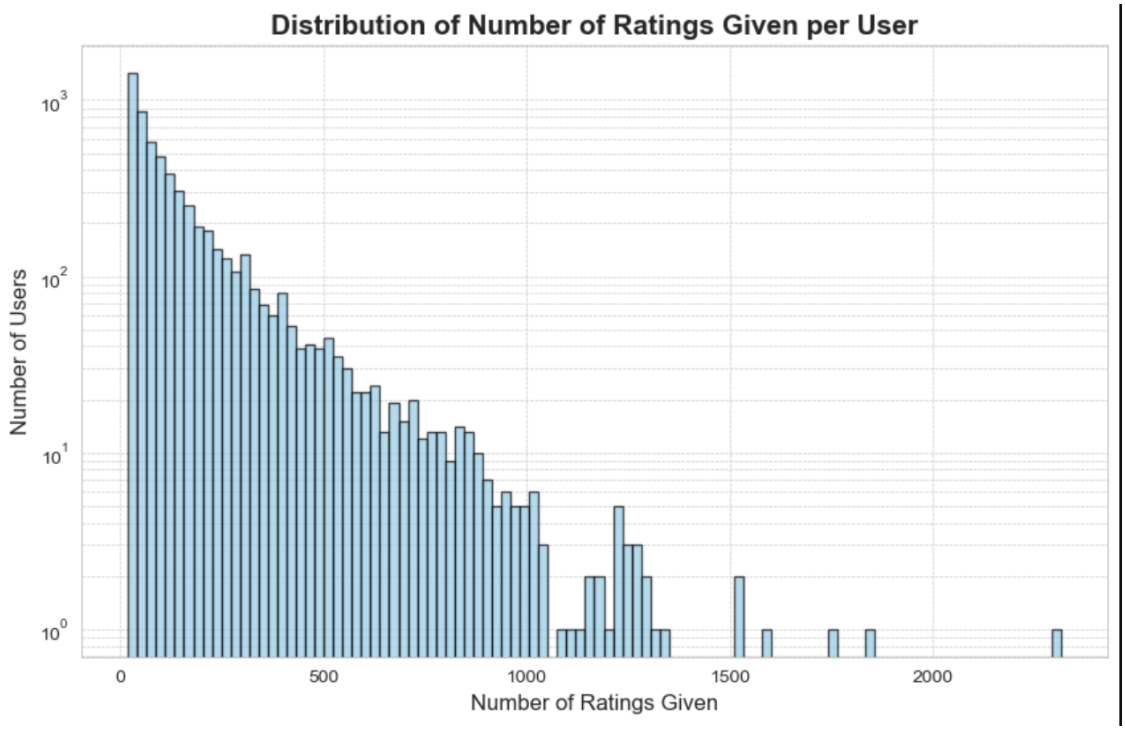

Figure A3: Distribution of Number of Ratings Given per User (Log Scale).

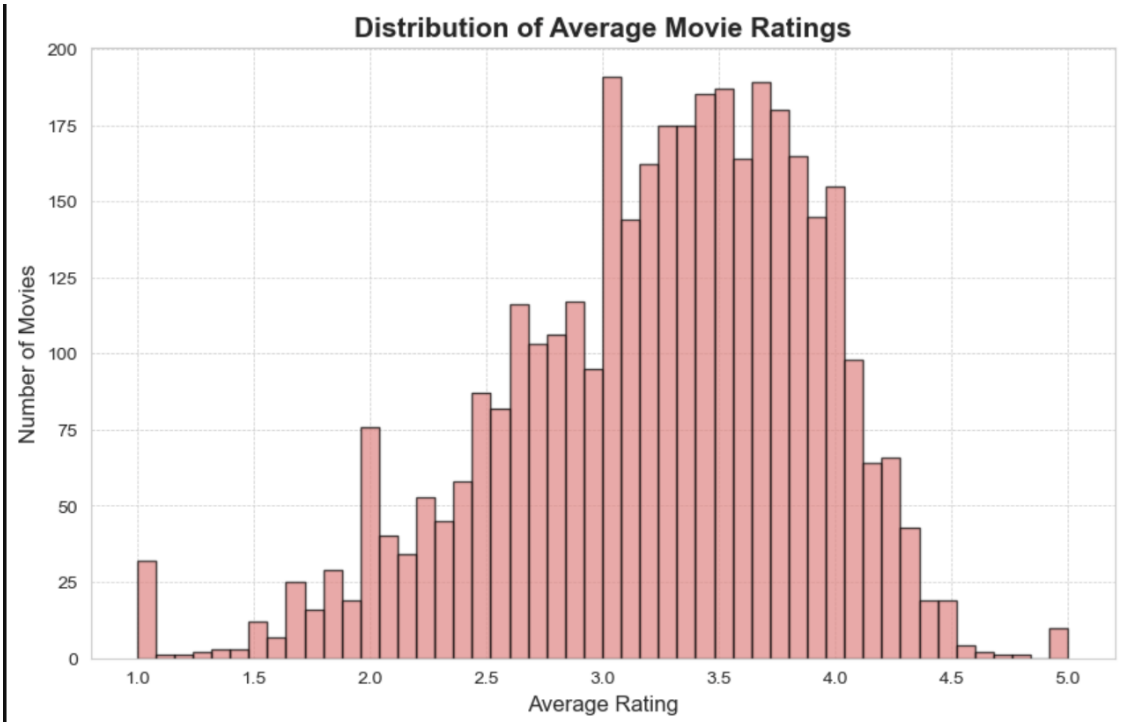

Figure A4: Distribution of Average Movie Ratings (Calculated per Movie).

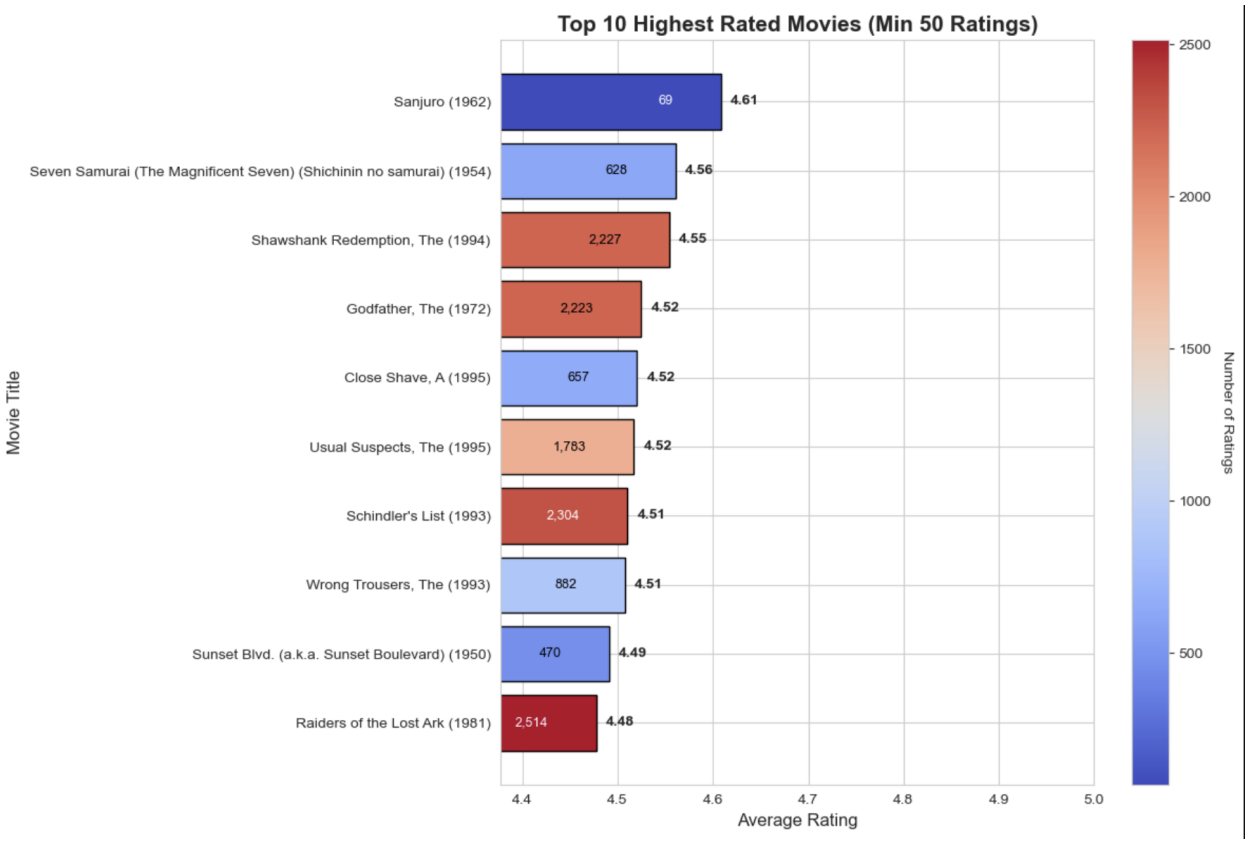

Figure A5: Top 10 Highest Rated Movies (Min 50 Ratings).

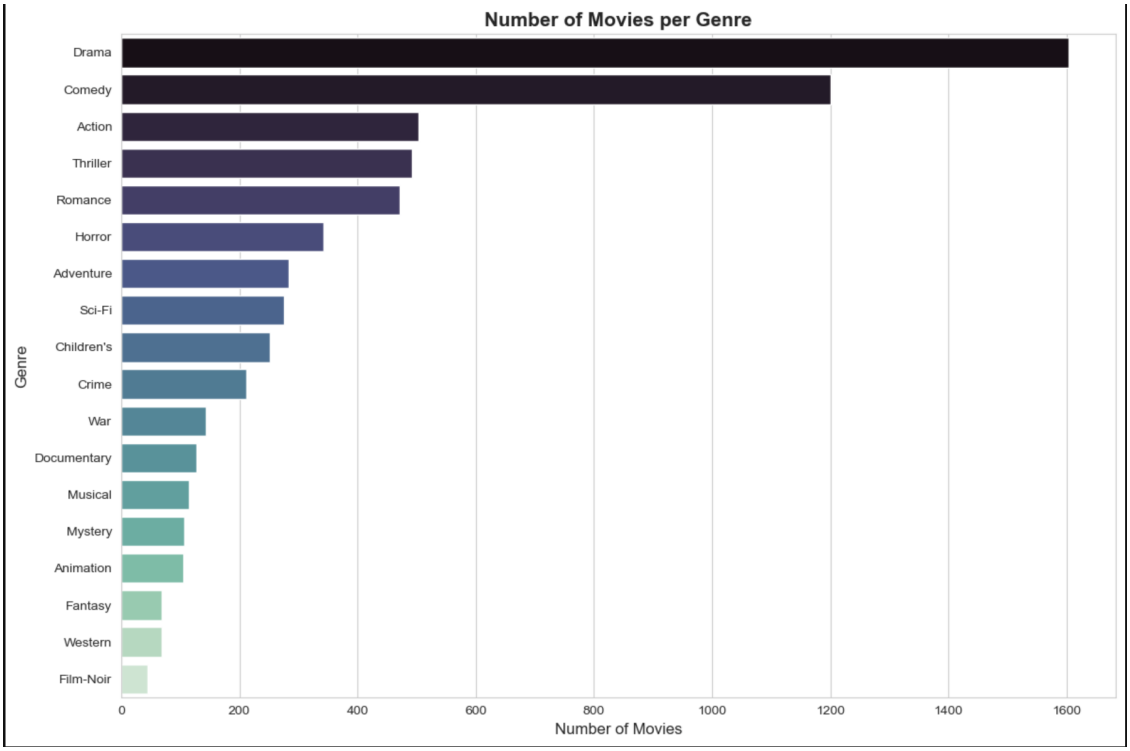

Figure A6: Number of Movies per Genre.

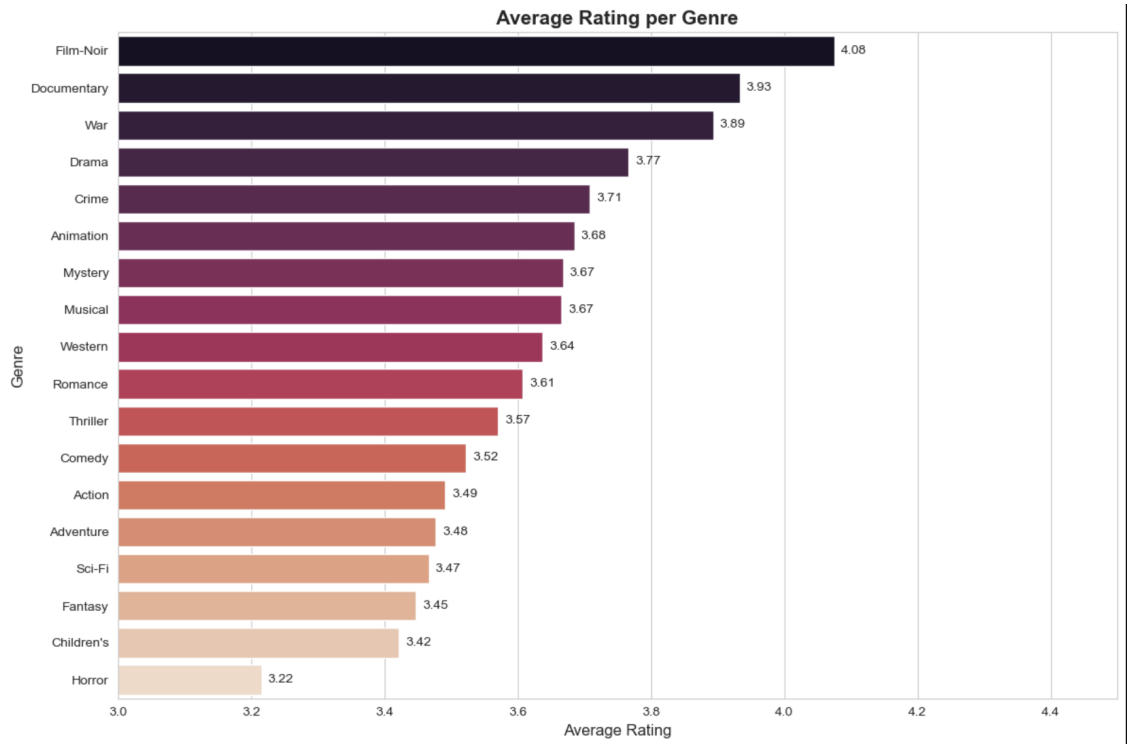

Figure A7: Average Rating per Genre.

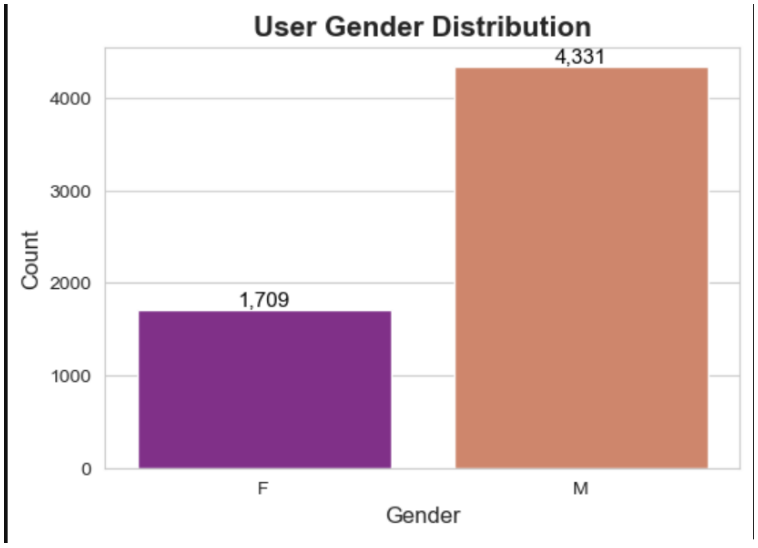

Figure A8: User Gender Distribution.

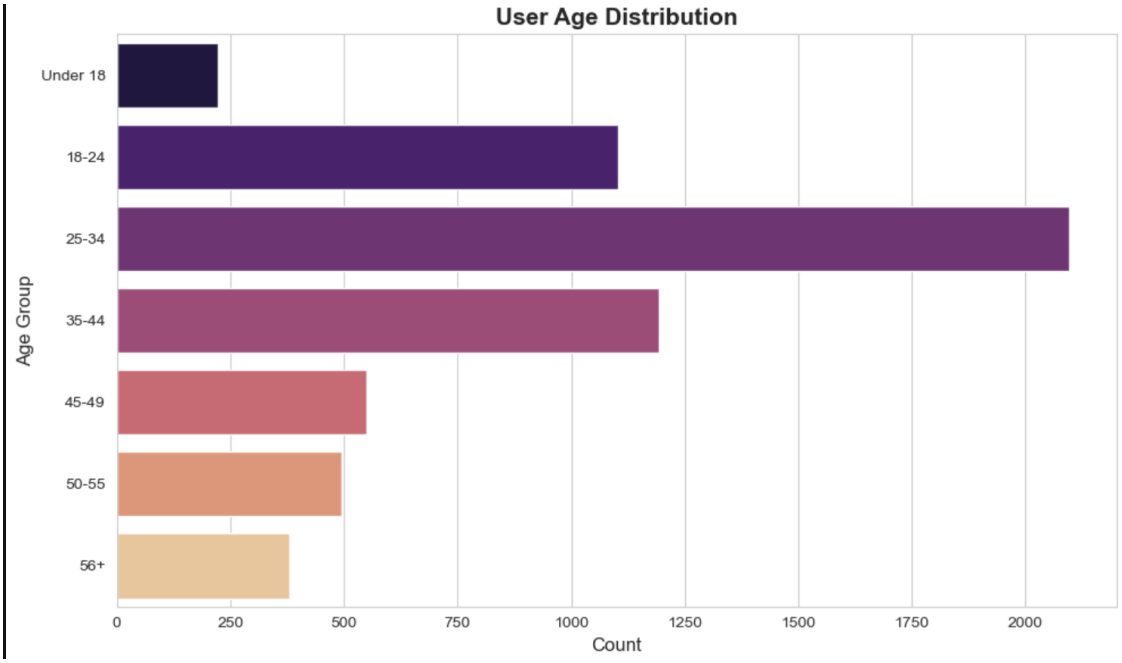

Figure A9: User Age Group Distribution.

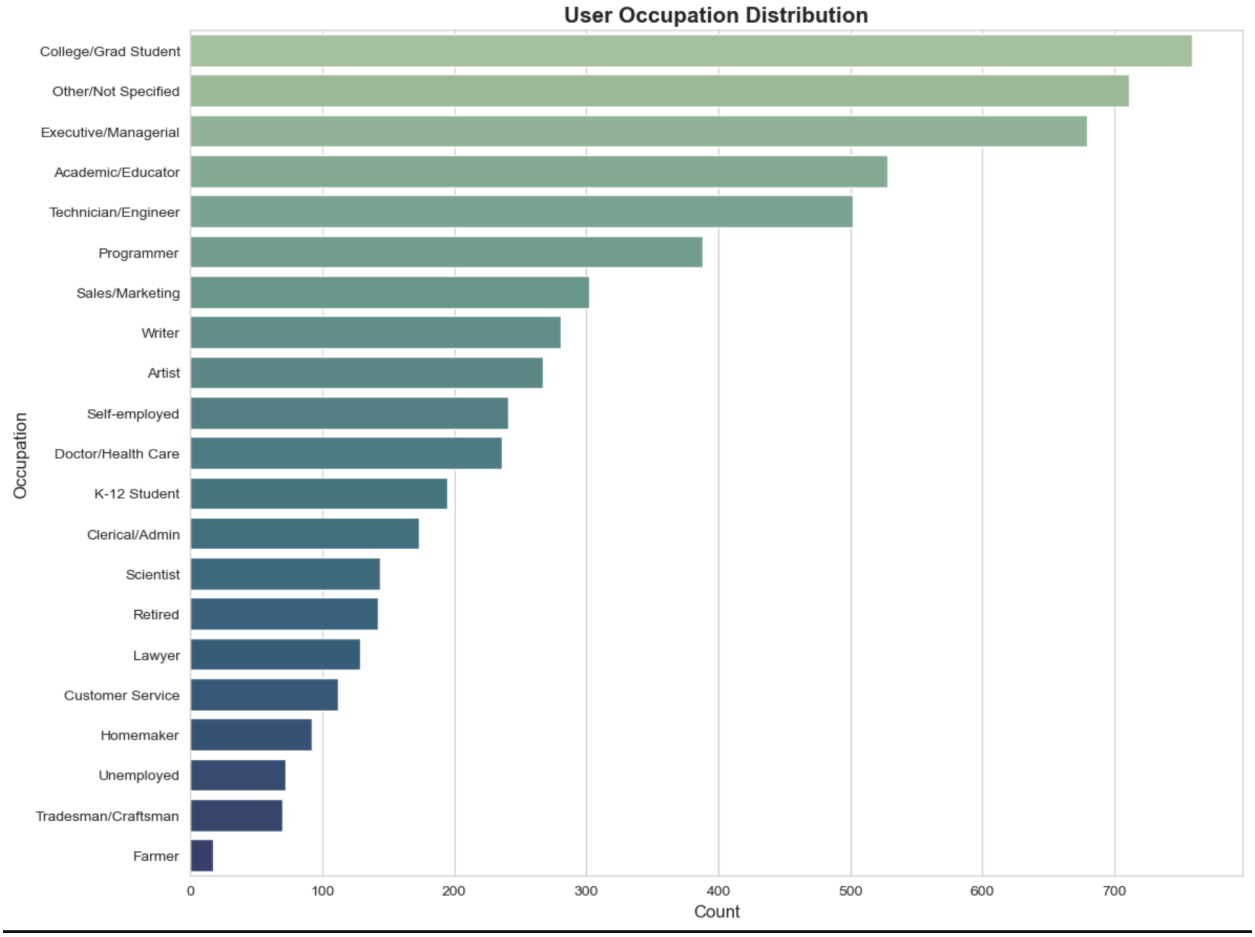

Figure A10: User Occupation Distribution.

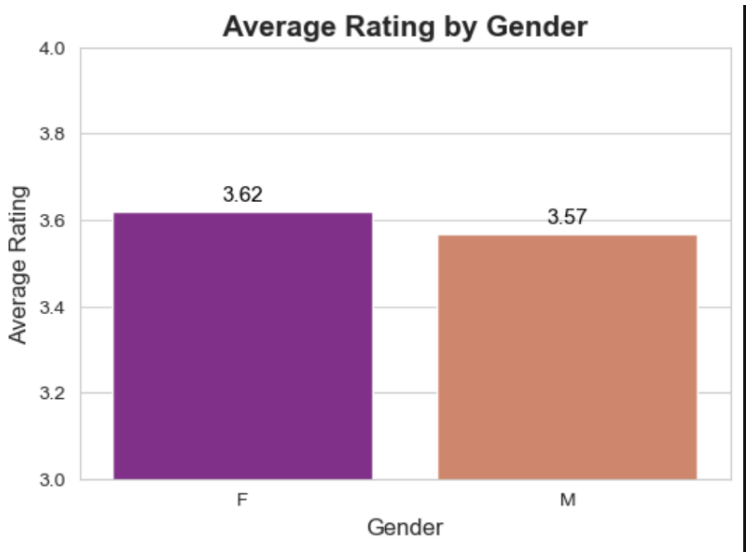

Figure A11: Average Rating by Gender.

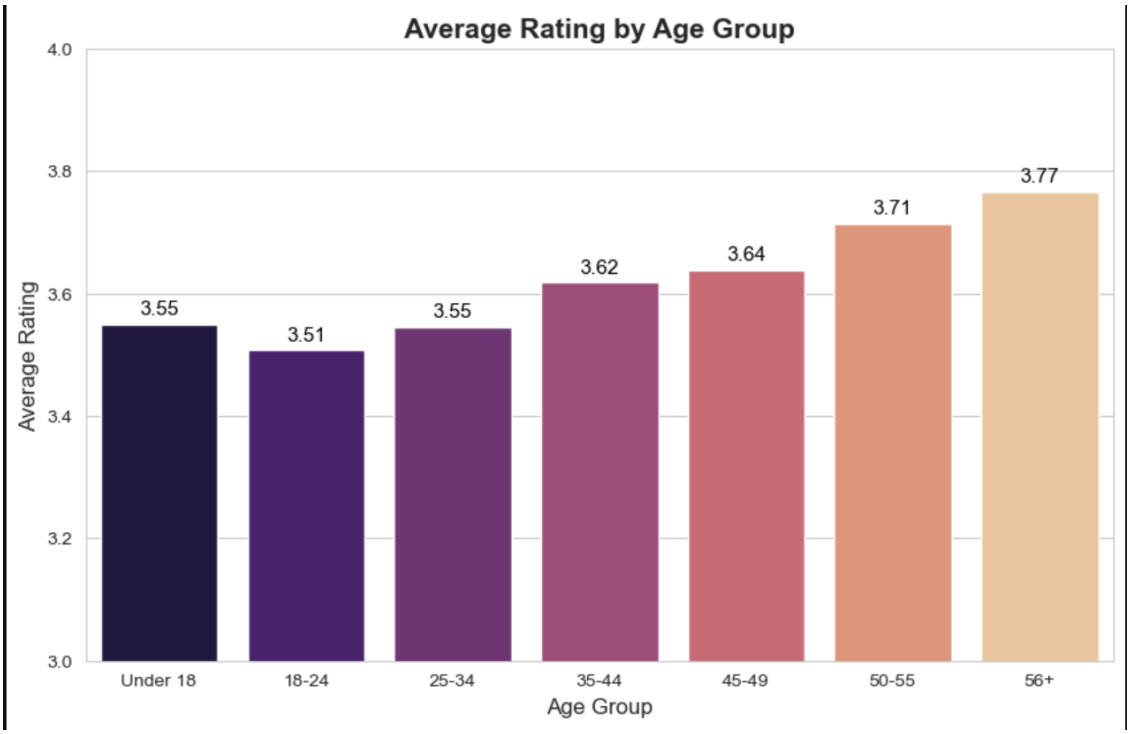

Figure A12: Average Rating by Age Group.

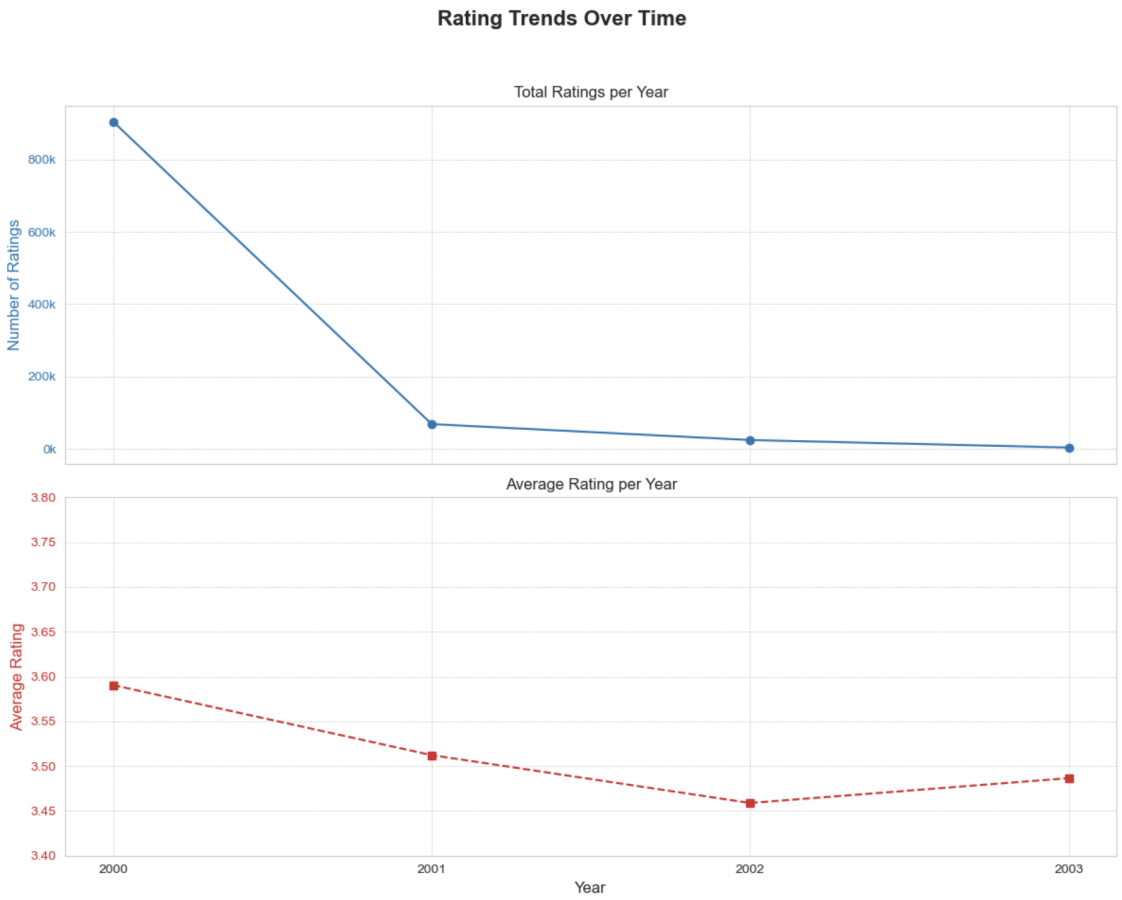

Figure A13: Rating Trends Over Time (Total and Average Rating per Year).

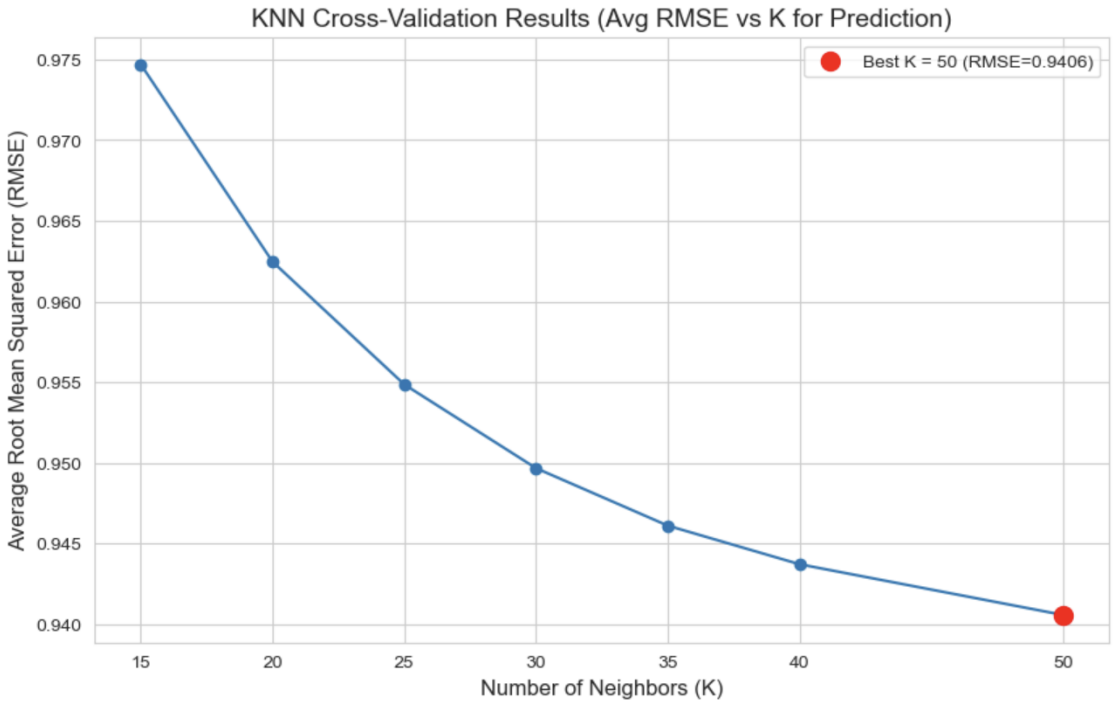

Figure A14: KNN Cross-Validation Results: Average RMSE vs. K.

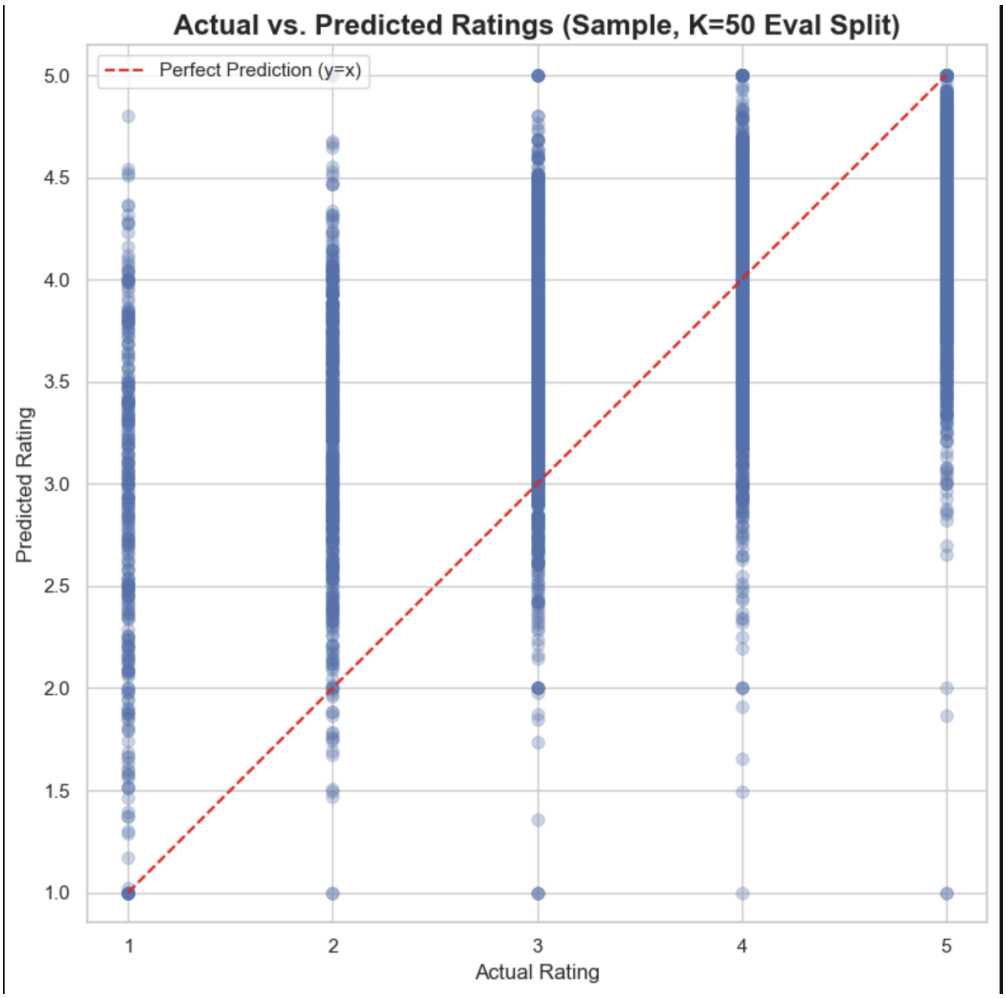

Figure A15: Actual vs. Predicted Ratings (Test Set, K=50).

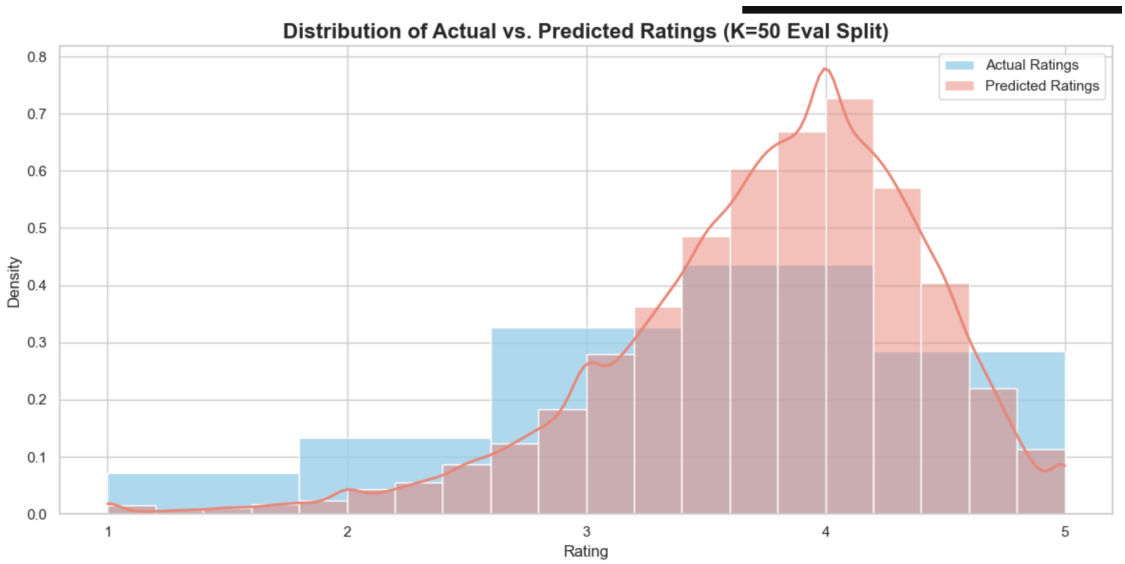

Figure A16: Distribution: Actual vs. Predicted Ratings (Test Set, K=50).

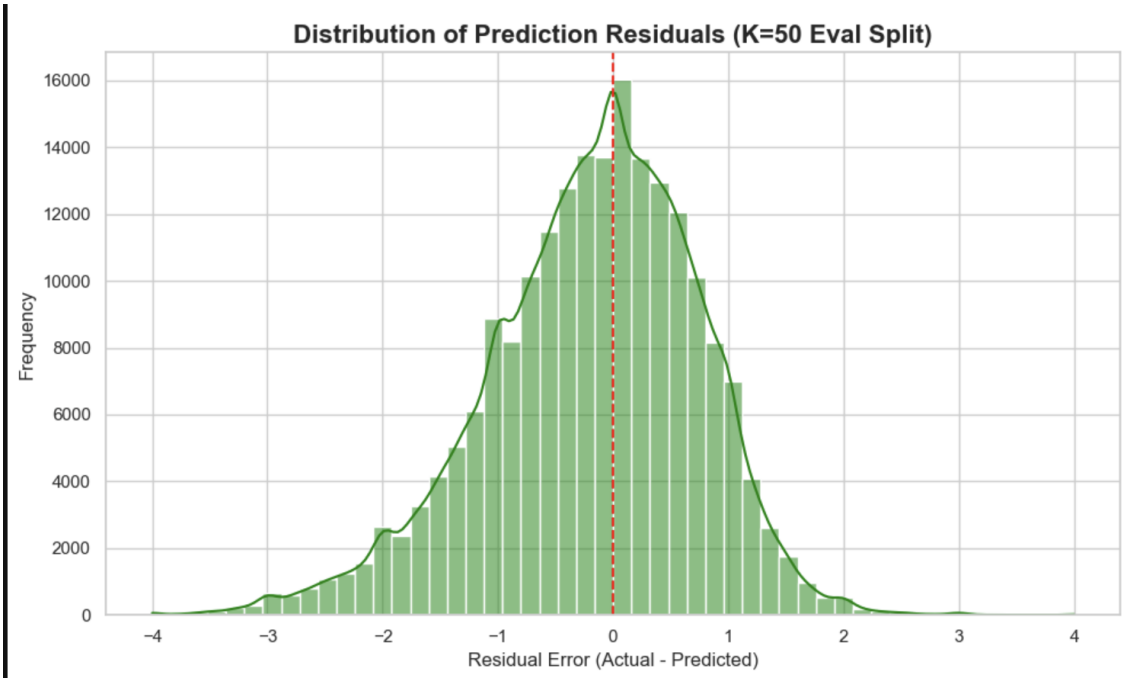

Figure A17: Distribution of Residuals (Test Set, K=50).

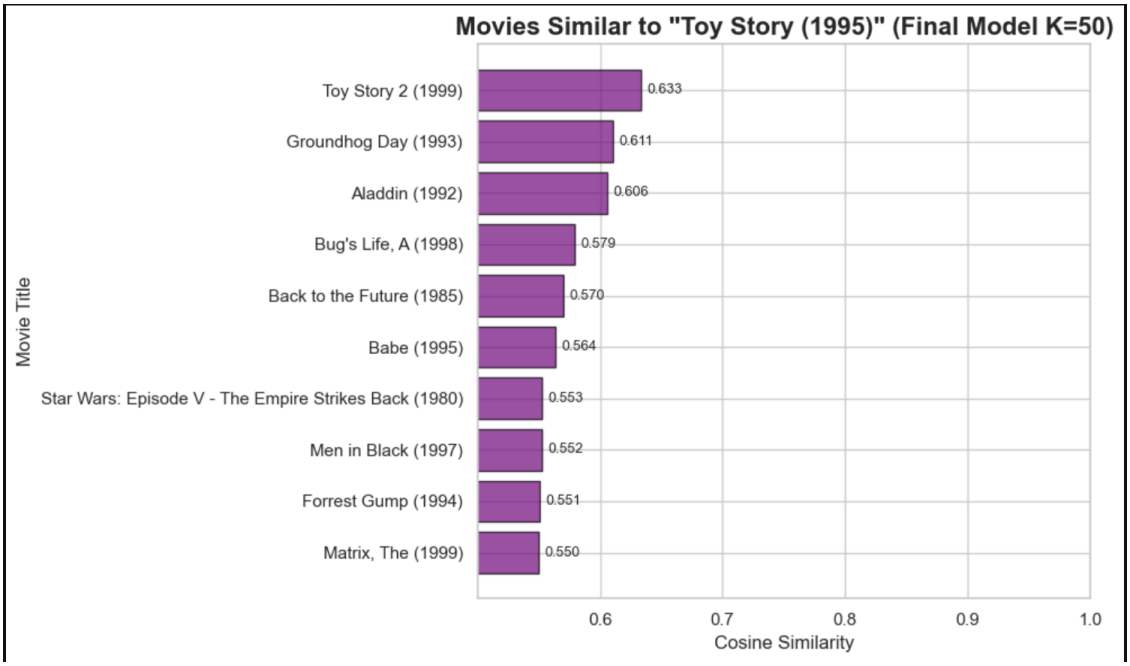

Figure A18: Movies Similar to *Toy Story (1995)* (K=50).

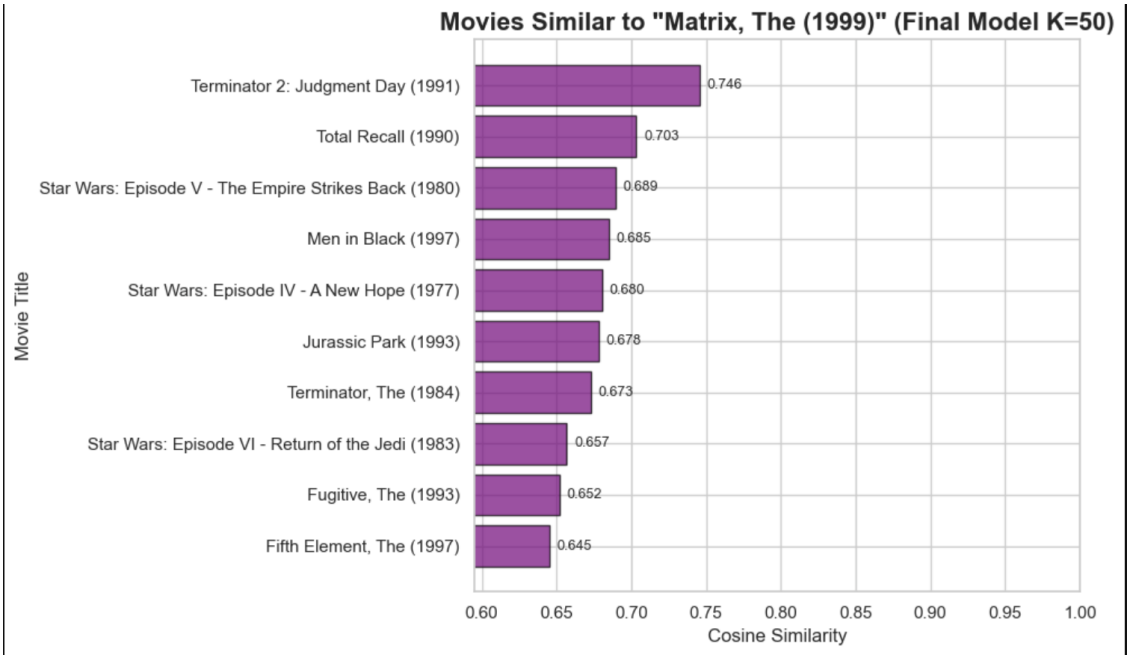

Figure A19: Movies Similar to *The Matrix (1999)* (K=50).

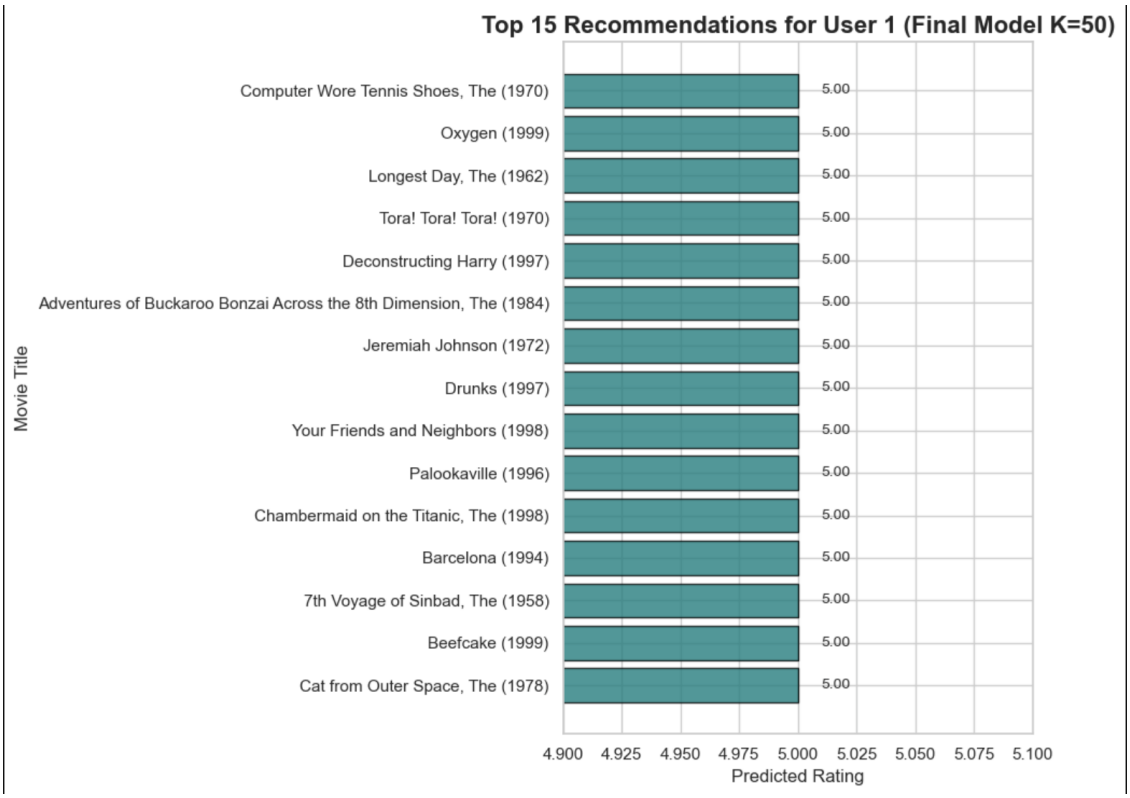

Figure A20: Top 10 Recommendations for User 1 (K=50).

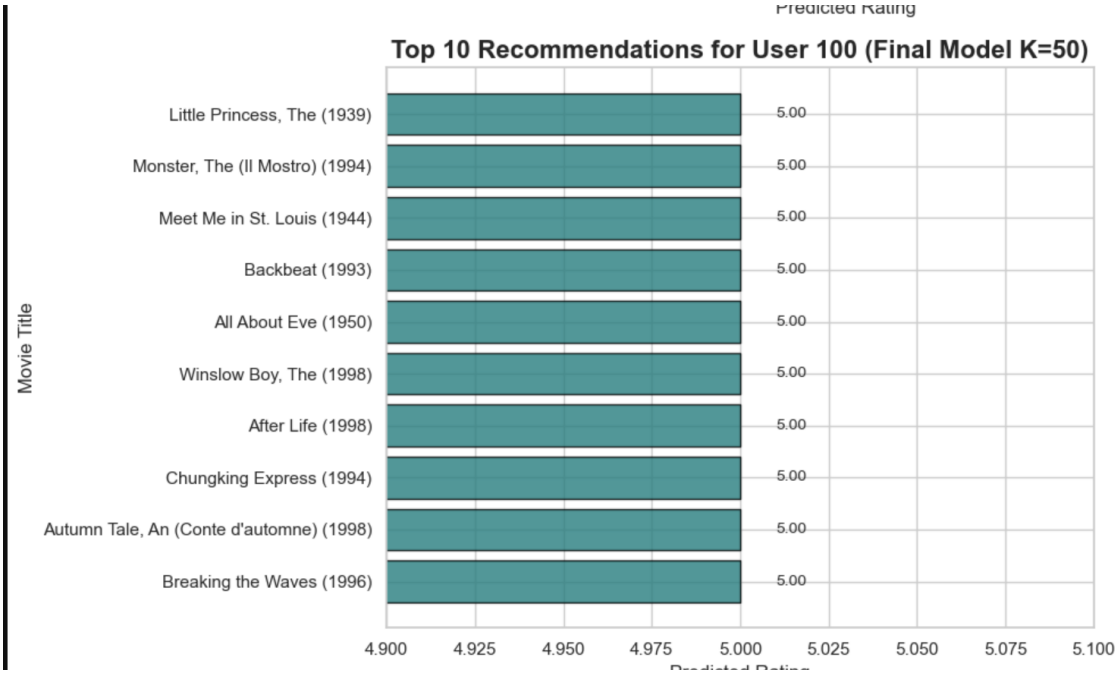

Figure A21: Top 10 Recommendations for User 100 (K=50).

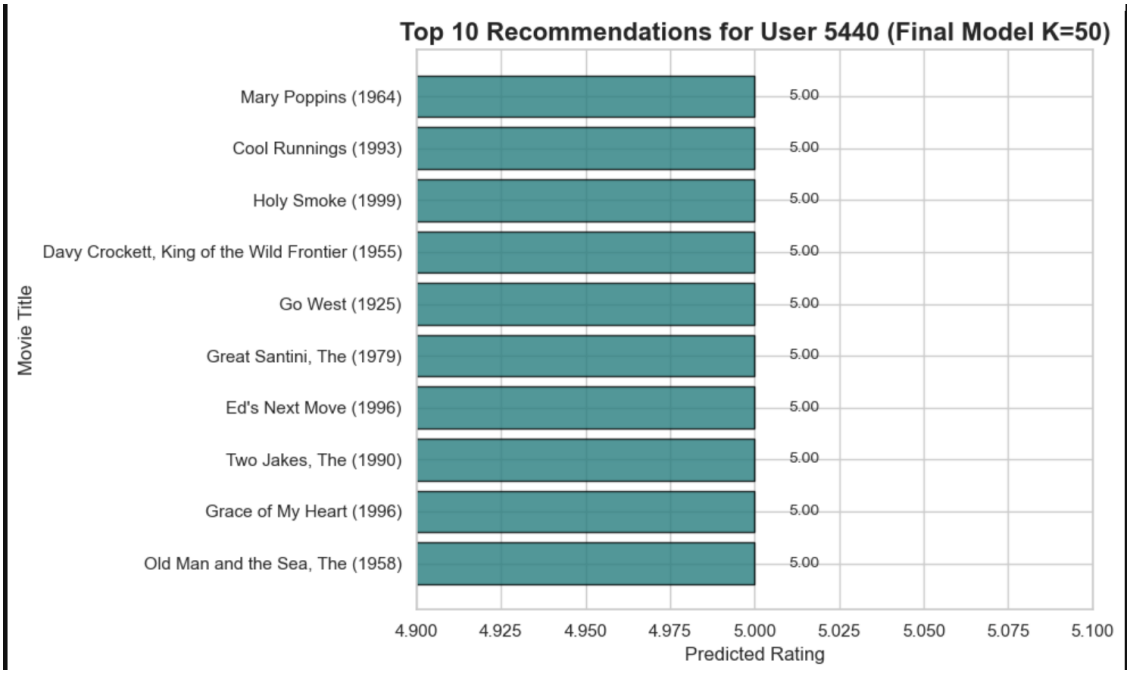

Figure A22: Top 10 Recommendations for User 5440 (K=50).